Introduction

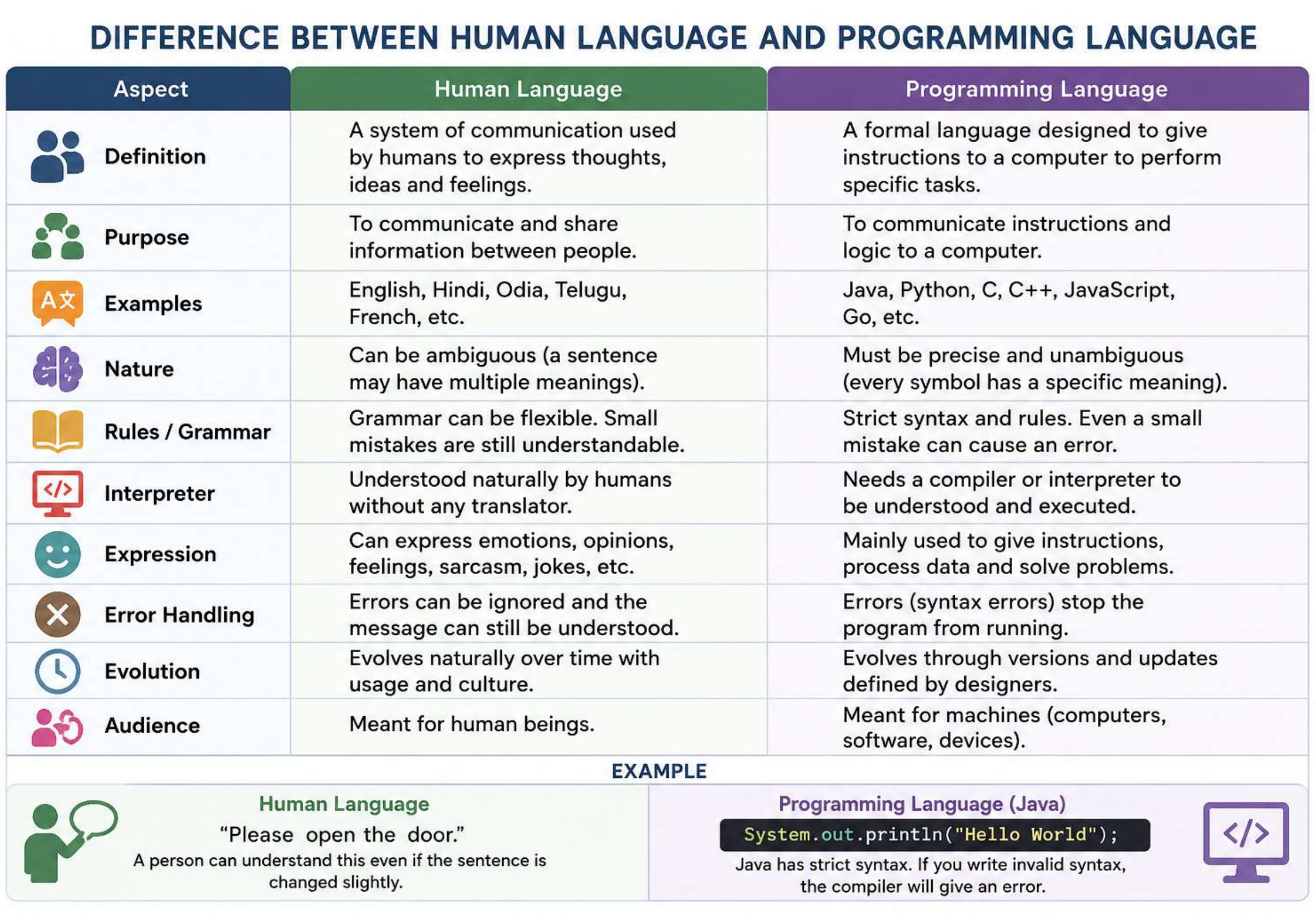
What is Language ?

ans) Language is a way of communicating with others using words, sounds, signs, or symbols.

Human language → communicates between people
Programming language → communicates instructions to computers

Examples

English – spoken/written language
Odia – spoken/written language
Hindi – spoken/written language
Java – programming language
Python – programming language



DIFFERENCE BETWEEN HUMAN LANGUAGE AND PROGRAMMING LANGUAGE

Aspect	Human Language	Programming Language
Definition	A system of communication used by humans to express thoughts, ideas and feelings.	A formal language designed to give instructions to a computer to perform specific tasks.
Purpose	To communicate and share information between people.	To communicate instructions and logic to a computer.
Examples	English, Hindi, Odia, Telugu, French, etc.	Java, Python, C, C++, JavaScript, Go, etc.
Nature	Can be ambiguous (a sentence may have multiple meanings).	Must be precise and unambiguous (every symbol has a specific meaning).
Rules / Grammar	Grammar can be flexible. Small mistakes are still understandable.	Strict syntax and rules. Even a small mistake can cause an error.
Interpreter	Understood naturally by humans without any translator.	Needs a compiler or interpreter to be understood and executed.
Expression	Can express emotions, opinions, feelings, sarcasm, jokes, etc.	Mainly used to give instructions, process data and solve problems.
Error Handling	Errors can be ignored and the message can still be understood.	Errors (syntax errors) stop the program from running.
Evolution	Evolves naturally over time with usage and culture.	Evolves through versions and updates defined by designers.
Audience	Meant for human beings.	Meant for machines (computers, software, devices).

EXAMPLE

Human Language

"Please open the door."

A person can understand this even if the sentence is changed slightly.

Programming Language (Java)

```
System.out.println("Hello World");
```

Java has strict syntax. If you write invalid syntax, the compiler will give an error.

Java

What is java ?

ans)Java is a high-level, object-oriented, class-based programming language used to develop different types of applications.

Java was developed by James Gosling and his team at Sun Microsystems in the early 1990s. It was initially designed for developing software for consumer electronic devices and later became widely used for Internet applications. The first official version of Java, JDK 1.0, was released on January 23, 1996.

Java is widely used for enterprise applications, backend development, web applications, microservices, and distributed systems.

Java is influenced by and syntactically similar to C and C++. Its syntax is largely derived from C/C++, but Java was designed to be simpler, more secure, and easier to use. Therefore, Java does not support some features that can make C/C++ more complex or error-prone, such as explicit pointers, structures, unions, operator overloading, and multiple inheritance through classes.

Why Java ?

==========

Java was developed to provide a simple, secure, robust, and platform-independent programming environment. Java became especially popular for Internet and network applications because of its platform independence and built-in networking support.

Unlike traditional compiled languages such as C and C++, Java source code is compiled into platform-independent bytecode, which can run on any system that has a compatible JVM. This follows the principle of "Write Once, Run Anywhere."

Important features of Java

======================

1 -> Simple – relatively easy compared with lower-level languages.
2 -> Object-Oriented – supports classes, objects, inheritance, polymorphism, abstraction, and encapsulation.
3 -> Platform Independent – bytecode can run on any system having a compatible JVM.
4 -> Secure – provides several security mechanisms.
5 -> Robust – strong memory management and exception handling.
6 -> Multithreaded – supports running multiple tasks concurrently.
7 -> Portable – Java programs can be moved between platforms.
8 -> High Performance – JVM uses technologies such as JIT compilation to improve execution speed.
9 -> Distributed – provides APIs useful for network/distributed applications.
10 -> Automatic Memory Management – Java uses Garbage Collection.

Where we use Java ?

==================

ans) we can use java in-

- 🖥️ Backend applications
- 🌐 Web applications
- ☁️ Cloud applications
- 🔗 Microservices
- 📱 Android development
- 🏦 Banking and enterprise applications
- 🛒 E-commerce applications
- 🔄 Distributed systems

(James Gosling)

(Company)

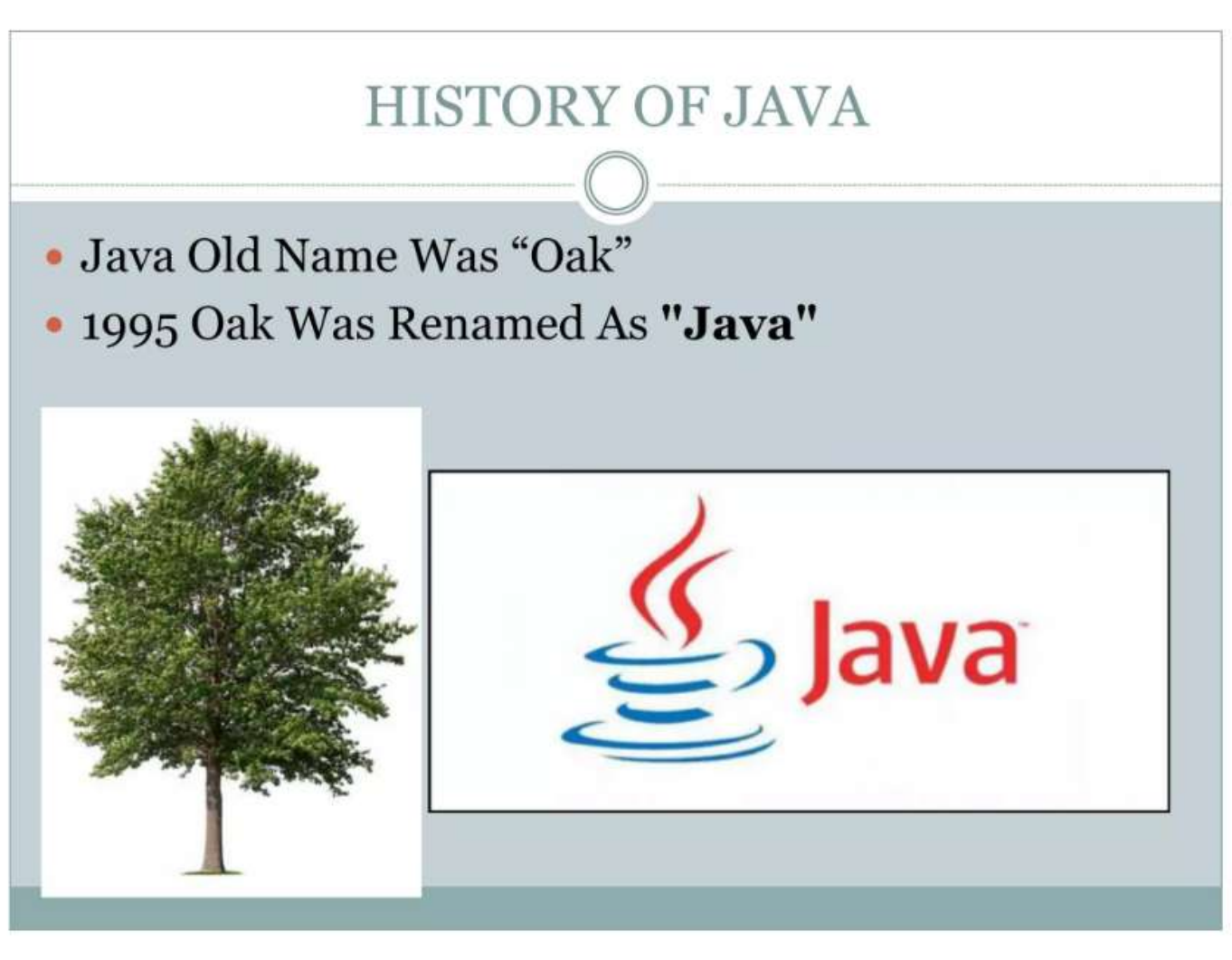
☕ History of Java — How Java Was Developed and Named

===

Java was developed at Sun Microsystems by a team called the Green Team.

The main person behind Java was James Gosling, along with engineers such as Mike Sheridan and Patrick Naughton.

The project started around 1991.

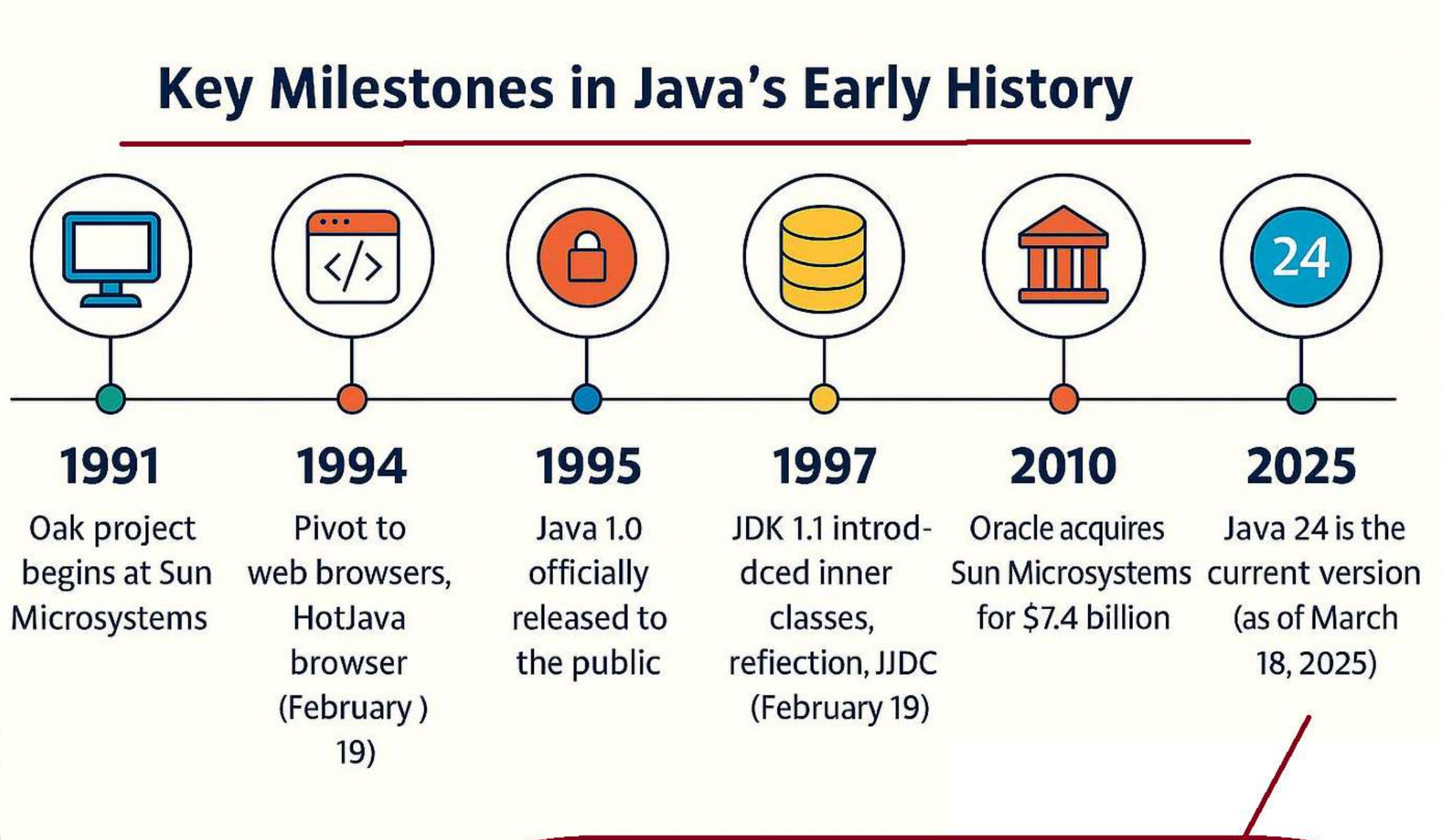


HISTORY OF JAVA

- Java Old Name Was "Oak"
- 1995 Oak Was Renamed As **"Java"**

Why name was change ?

They were unable to register this programming language with name OAK, because already some other product is registered with the same name. So they renamed to Java.



Key Milestones in Java's Early History

1991 Oak project begins at Sun Microsystems

1994 Pivot to web browsers, HotJava browser (February) 19)

1995 Java 1.0 officially released to the public

1997 JDK 1.1 introddced inner classes, refiection, JJDC (February 19)

2010 Oracle acquires Sun Microsystems for $7.4 billion

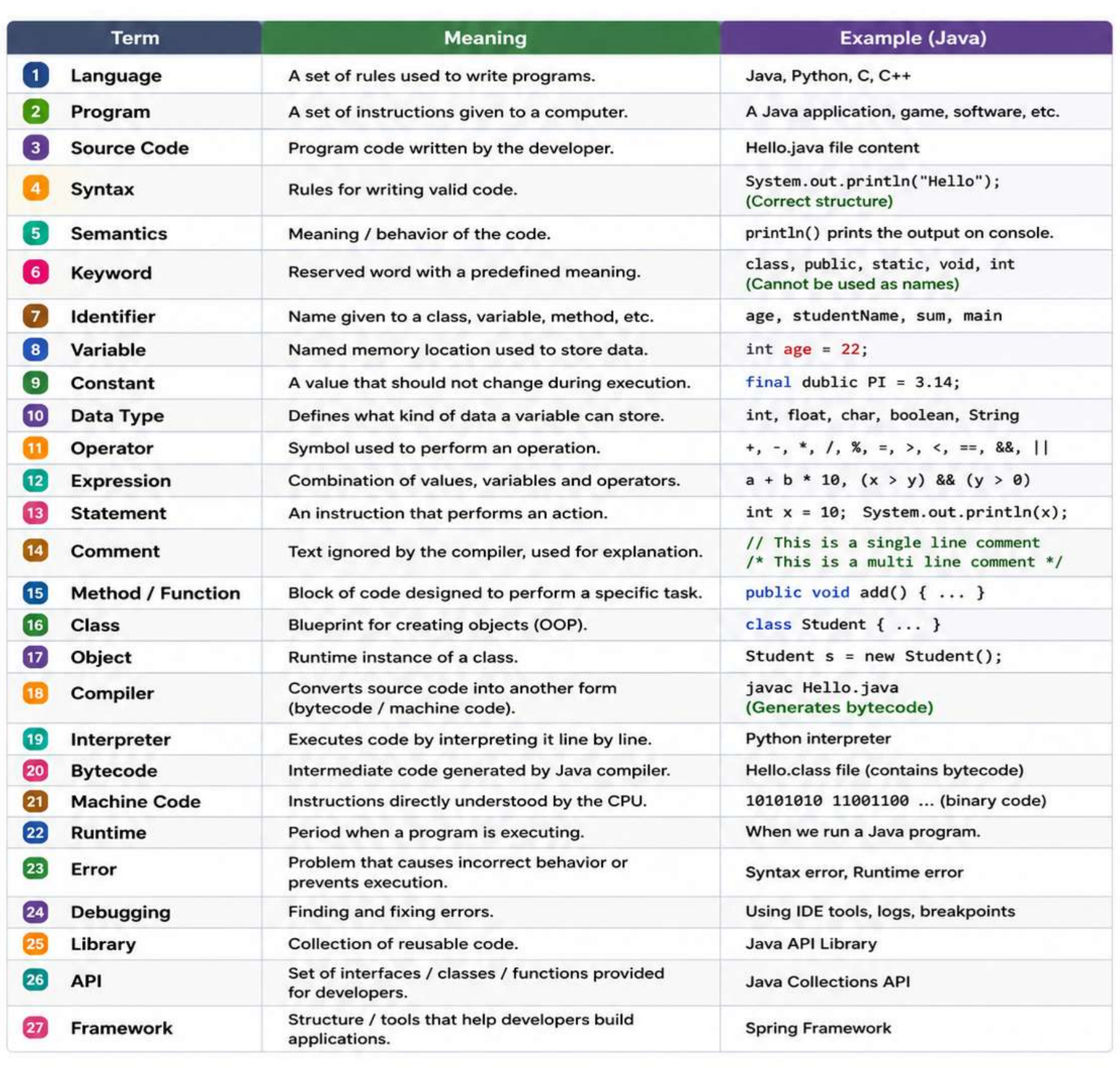
2025 Java 24 is the current version (as of March 18, 2025)

but now current version is not 24

java 26(JDK 26.0.2.1) is current vesion but

Oracle confirms that Java 25 is an LTS release, while Java 26 is the latest feature release; the next planned LTS is Java 29 in September 2027.

LTS= Long Term Support



TERMINOLOGY USED IN PROGRAMMING LANGUAGES

Term	Meaning	Example (Java)
1 **Language**	A set of rules used to write programs.	Java, Python, C, C++
2 **Program**	A set of instructions given to a computer.	A Java application, game, software, etc.
3 **Source Code**	Program code written by the developer.	Hello.java file content
4 **Syntax**	Rules for writing valid code.	`System.out.println("Hello");` (Correct structure)
5 **Semantics**	Meaning / behavior of the code.	`println()` prints the output on console.
6 **Keyword**	Reserved word with a predefined meaning.	`class, public, static, void, int` (Cannot be used as names)
7 **Identifier**	Name given to a class, variable, method, etc.	`age, studentName, sum, main`
8 **Variable**	Named memory location used to store data.	`int age = 22;`
9 **Constant**	A value that should not change during execution.	`final dublic PI = 3.14;`
10 **Data Type**	Defines what kind of data a variable can store.	`int, float, char, boolean, String`
11 **Operator**	Symbol used to perform an operation.	`+, -, *, /, %, =, >, <, ==, &&, ||`
12 **Expression**	Combination of values, variables and operators.	`a + b * 10, (x > y) && (y > 0)`
13 **Statement**	An instruction that performs an action.	`int x = 10;  System.out.println(x);`
14 **Comment**	Text ignored by the compiler, used for explanation.	`// This is a single line comment` `/* This is a multi line comment */`
15 **Method / Function**	Block of code designed to perform a specific task.	`public void add() { ... }`
16 **Class**	Blueprint for creating objects (OOP).	`class Student { ... }`
17 **Object**	Runtime instance of a class.	`Student s = new Student();`
18 **Compiler**	Converts source code into another form (bytecode / machine code).	`javac Hello.java` (Generates bytecode)
19 **Interpreter**	Executes code by interpreting it line by line.	Python interpreter
20 **Bytecode**	Intermediate code generated by Java compiler.	Hello.class file (contains bytecode)
21 **Machine Code**	Instructions directly understood by the CPU.	`10101010 11001100` ... (binary code)
22 **Runtime**	Period when a program is executing.	When we run a Java program.
23 **Error**	Problem that causes incorrect behavior or prevents execution.	Syntax error, Runtime error
24 **Debugging**	Finding and fixing errors.	Using IDE tools, logs, breakpoints
25 **Library**	Collection of reusable code.	Java API Library
26 **API**	Set of interfaces / classes / functions provided for developers.	Java Collections API
27 **Framework**	Structure / tools that help developers build applications.	Spring Framework